Razorpay POS
Integration Solution Doc

- → POS Bridge
- → DQR
- → Android SDK Solution

RAZORPAY POS'S ANDROID SDK

RAZORPAY POS SDK INTRODUCTION

Razorpay POS is a digital payments platform that makes your payment acceptance simple and configurable. This Universal Payment Acceptance platform is a smart, simple end-to-end payments platform which is unique in its vision and architecture.
It can be deeply integrated with any enterprise system/sales channel to offer a unified and seamless payments experience to the end customer.

WHY RAZORPAY POS SDK?

- Enterprises who have sales and collection application already built and would like to accept digital Payments via the application. Razorpay POS's android SDKs are available to be plugged in to the existing application for payments acceptance.
- This integration is fast and easy, and typically takes around 2-3 days of time to complete.
- Enterprises can maintain one single application for business process as well as payments and can future proof their payments capabilities with easy enablement of newer modes of payments.
- **NOTE:** Razorpay POS android SDK will only support Android version 5.0 and above. This is due to PCI DSS compliance.

WHAT DOES RAZORPAY POS ANDROID SDK DO?

Razorpay POS SDK enables enterprises field application with for universal payments acceptance. The SDK helps you to integrate with the Service Application of Razorpay by calling an API (Which can be initiated via a single “Pay” button on your application), Razorpay POS Service Application manages all interactions between the Card, Device and the , Razorpay POS server, and encapsulates a smooth user experience during the entire payment cycle. The below figure showcases the flow of payment using Razorpay POS SDK.

GETTING STARTED WITH ANDROID SDK INTEGRATION

We recommend that you get started with integration in the demo environment, validate the flow and use cases on your application and then move to the Production environment.
You will receive the below details from Razorpay POS Solution Consultant for integration with the demo environment.

- Appkey for Authentication
- Credentials to access the demo portal

If you haven't received these details yet, please share the below details to respective Sales or Solution Consultant for an account to be created in the Demo Environment.

- Your Organisation's Name
- Your Organisation's Address
- Contact Number & Email ID

Please note: Devices which are in debug mode have a watermark - "DEBUG only, Not for Commercial", at the bottom right corner of the screen.

01 Merchant agent logs into the android app integrated with Razorpay SDK

02 Merchant app fetches the detail from merchant server

03 Payment request is sent from SDK to Razorpay server

4A Payment horization

4B Authentication & validation

05 Payment response sent back from Razorpay server to app

06 Updated merchant CRM with payment details

Customer server

Payment authorization

Issuer Bank

HOW TO INCLUDE PAYMENTS SDK IN YOUR ANDROID APPLICATION

Razorpay POS payments SDK can be integrated with different types of android SDK such as Native, react native, Cordova, etc. Integrating the SDK will allow you to access Razorpay POS services in your application.
For step-by-step guide with code and other details, please refer to our GitHub portal here . You will find the following details there :

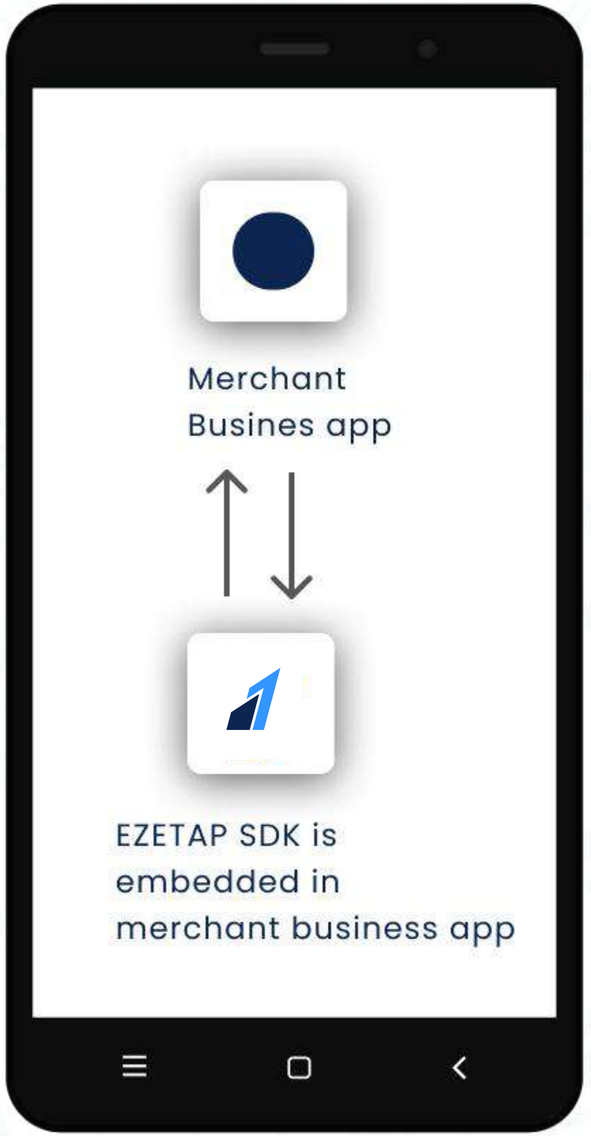
- What you need for Integration
- Sample App
- Documentation to create Cordova plugin for Android
- Sample code for **Native** / **Cordova** / **React Native**

HOW DOES THE POS ANDROID SDK WORK?

- Include the SDK in your mobile application to collect digital payments.
- SDK interfaces with Razorpay Service Application, which has a pet name Service App.
- The SDK inspects the availability of Service App on Android device and if not present it will be installed at run-time.
- **Service App** interfaces with the Card Device (in card of card payments) and Razorpay POS Servers to finish payment processing and notifies the final status to SDK/Client App.

For more details, please refer to our **portal**. You will find the following details

- Invoking Razorpay POS API
- Handling Responses
- Sample code to fetch the response
- Response Structure
- Sample Response



01 An API call launches the service app

02 Service app communicates with Razorpay

Server

APK

Service app

04 Service app finishes the txn processing & notifies the SDK

03 Server sends the request back to service app

STEP 1 : INITIALISING THE SDK

The first API that needs to be integrated with is the “Initialize” API, this API performs the following 3 key activities and 1 optional activity:

- Initializes the SDK with global configuration settings
- Connects to the appropriate Razorpay POS server based on Application mode (AppMode) = DEMO/ PROD
- Connects to the appropriate merchant account, this depends on the app key entered
- Optionally invokes prepare Device to initialize the device (card reader) with the updated encryption keys from the corresponding bank

Initialize is the first method to be called. It is recommended to call this method after the user logs in to your application or (if login is not available) when he reaches the home screen of your application.
For step-by-step guide to initialize the SDK, please refer to our portal here -
You will find the following details:

- How to call initialize API
- How to prepare input for initialize API
- How to invoke the initialize API
- How to handle the response of Initialize API
- Sample Request and Response

STEP 2: TRANSACTION PAYMENT RESPONSE VIA AND SERVER TO SERVER CALL-BACK

The transaction response can be provisioned back to the client application using the SDK itself using onResultActivity method. However, if the client is looking for transaction response provisioning in some other way, Razorpay POS can provide a sever to server call back for each and every successful transaction. Also there will be a retrial mechanism to retry posting up to 3 times if http status 200 is not received as an acknowledgement in the previous attempts.
Client will have to provide an endpoint on which the details will be sent for successful transactions.

STEP 3: UNIVERSAL PAY API FOR PAYMENT TRANSACTION

Razorpay POS has a universal pay API through which all the payment modes (that is enabled for the merchant) can be invoked through a single API call. With this API there will be no need of calling the individual methods for different payment modes (like Card, Remote Pay, QR etc).
Please refer to our portal here -
You will find the following details:

- How to prepare Input for the Pay API
- How to invoke Pay API
- How to handle the response of Pay API
- Basic structure of Pay API response
- Sample Pay API response for payment by Card
- How to handle payment failures

STEP 4: CARD PAYMENT API FOR PAYMENT VIA CARD

In addition to the universal Pay API described above, Card Payment API is also available for card-based payment transaction.
Please refer to our portal here -
You will find the following details:

- How to invoke Card Payment API
- JSON request
- Sample Request
- Handling the payment response
- Sample Response

STEP 5: HOW TO IDENTIFY A SUCCESSFUL CARD TRANSACTION

For Card transaction, please rely on the Status Field in the Pay API or Card Payment API response to identify a successful transaction

Status	What it means
Authorized	Transaction has been successfully executed
Failed	Transaction has not been executed and has failed; the money won’t be deducted in this scenario
Voided	The transaction was authorized, and which is now has been voided.
Refunded	The transaction was completed and after which it was refunded

APIS FOR OTHER PAYMENT MODES SUCH AS UPI, QR, ETC.

APIs for other payment modes such as UPI, Cash, Cheque, etc. are also available. Please refer below for details of all such APIs and respective links to our portal.

API for payment menthods	Link to portal
Cash	**Link**
Cheque	**Link**
UPI	**Link**
Remote Pay	**Link**
QR Code	**Link**
Wallet	**Link**

Upon clicking the link provided in the table above, you will find below information

- How to prepare input for API
- How to invoke the API
- How to handle response of the API
- Sample Request & Response

ADDITIONAL APIS FOR OTHER OPERATIONS

Some other useful APIs that may be required for payment and related operations are also available. Please refer below for details of all such APIs and respective links to our portal.

API	Usage	Link to portal
Service Fee	To add Service Fee	**Link**
Accepting Meal Cards	To accept payment via meal cards	**Link**
Void Payment API	To process Refund on same day	**Link**
Check for Incomplete Payment	To check for the incomplete transaction	**Link**
Fetch Payment Details	To retrieve the details of a payment transaction	**Link**
Sending Receipts	To send e-receipts via SMS/e-mail	**Link**
Print Receipts	To print receipt/charge-slip/custom slip	**Link**
Close SDK	To exit from Razorpay POS SDK	**Link**

Upon clicking the link provided in the table above, you will find below information

- How to prepare input for API
- How to invoke the API
- How to handle response of the API
- Sample Request & Response

PRINT CUSTOM RECEIPTS, BILLS, INVOICE, IN ANY FORMAT

Razorpay POS provides custom print SDK which allows printing of custom bills, invoices receipts in addition to the chargeslip. Bills can be printed via the device in any format, layout of our choice.
Text Size & Font:

- Text Size: setTextSize(TypedValue.COMPLEX_UNIT_PX,15sp)
- font family: lato_bold

For sample code, SDK, please refer to the link.
The way to use print bitmap function is to arrange the details in xml format and then generate a bitmap of the same and print it further using the printer SDK.
Sample code for Bitmap Printing :

```
JSONObject jsonRequest = new JSONObject();
JSONObject jsonImageObj = new JSONObject();
try {
  img.buildDrawingCache();
  Bitmap bmap = img.getDrawingCache();
  String encodedImageData = getEncoded64ImageStringFromBitmap(bmap);
  // Building Image Object
  jsonImageObj.put("imageData", encodedImageData);
  jsonImageObj.put("imageType", "JPEG");
  jsonImageObj.put("height", "");// optional
  jsonImageObj.put("weight", "");// optional
  jsonRequest.put("image", jsonImageObj); // Pass this attribute when you have a valid captured
signature image
  EzeAPI.printBitmap(this, REQUEST_CODE_PRINT_BITMAP, jsonRequest);
} catch (JSONException e) {
  e.printStackTrace();
}
```

ERROR CODES

Typically, you will notice 4 types of errors during transactions:

- Common Errors
- Server Errors
- PG Errors
- SDK Errors

Full list of these error codes is available on our **portal** here .
Please refer to this for prominent error codes and their definitions.

STEPS FOR GO LIVE

Testing the Integration

A detailed testing scenario is crucial and critical step before the roll-out to production, you can test all payment related integration features End-to-End including failure cases, payment gateways errors etc. To facilitate this on our demo environment we do simulation of error mapped to certain amounts. Below you can find the amounts and mapped error cases.

Best Practice: Razorpay POS recommends testing at least 50% of these scenarios to conclude your UAT

Please refer to our portal **here**.
You will find the details of amounts to be passed to simulate a specific error.

GO-LIVE

Once you closed your integration and testing on our DEMO environment, you should consult the Razorpay POS team and get your integration verified. Post we have confirmed and tested the integration and various scenarios; you will be eligible to go live. Here are the steps you need to follow for Going-live:

- Procure a Production device via the bank (only for card payments)
- Inform the Razorpay POS once the device has been procured
- Razorpay POS team will provide you the Production App Key, add the Production App key in your initialise API request.
- Change the App mode to PROD, from DEMO

Best Practice: Razorpay POS recommends that you perform some test Re.1 Transactions and verify the transaction flow, portal information etc.

Razorpay POS

This document has been crafted to offer an in-depth exploration of the integration steps and artefacts required for Razorpay POS Integration.

For any additional inquiries, please feel free to contact us at pos-integrations@razorpay.com.